

1. STL stands for Standard Template Library.

STL is a collection of **ready-made classes, functions, and data structures** provided by C++ that help us store, process, and manipulate data efficiently.

Instead of creating common data structures and algorithms from scratch, we can use the components already provided by STL.

Simple Example

Suppose we want to store the marks of 5 students.

Without STL:

```
int marks[5] = {80, 75, 90, 65, 88};
```

An array has a fixed size.

With STL:

```
vector<int> marks = {80, 75, 90, 65, 88};
```

A `vector` can automatically grow or shrink when elements are added or removed.

In simple words

STL gives us ready-made tools for storing and processing data.

2. Why Do We Need STL?

Before STL, programmers often had to implement common data structures and algorithms themselves.

For example, suppose we want to sort:

```
50 20 40 10 30
```

Without using a ready-made algorithm, we would have to write a sorting algorithm ourselves.

With STL:

```
sort(numbers.begin(), numbers.end());
```

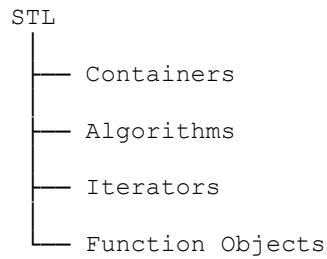
That's it.

So STL:

- reduces programming effort
- provides tested and reusable components
- makes programs shorter
- improves productivity
- provides efficient data structures and algorithms

3. Main Components of STL

The most important parts of STL are:



We will learn them one by one.

Containers

Used to **store data**.

Examples:

```
Vector, list  
Deque, stack  
Queue, priority_queue  
Set, map
```

Algorithms

Used to **perform operations on data**.

Examples:

```
sort(), find()  
reverse(), count()  
max(), min()
```

Iterators

Used to **move through/access elements of a container**.

Examples:

```
begin(), end()
```

Function Objects

Objects that can behave like functions and can be passed to algorithms.

For example:

```
greater<int>()
```

What is a Vector?

A `vector` is an STL container that works like a **dynamic array**.

The important word is **dynamic**.

A normal array:

```
int marks[5];
```

has a fixed size.

A vector can grow:

```
vector<int> marks;
```

We can keep adding elements.

Real-life analogy

Think of a classroom.

Initially:

```
Classroom  
[Student 1]
```

Another student joins:

```
[Student 1] [Student 2]
```

Another:

```
[Student 1] [Student 2] [Student 3]
```

The vector automatically manages its storage.

How to Create a Vector?

First include the vector header:

```
#include <vector>
```

Then:

```
vector<int> numbers;
```

Here:

- `vector` → STL container
- `<int>` → type of data
- `numbers` → vector name

Complete program:

```
#include <iostream>  
#include <vector>  
using namespace std;  
  
int main()  
{  
    vector<int> numbers;  
  
    return 0;  
}
```

```
}
```

Adding Elements — `push_back()`

To add an element at the end of a vector, use:

```
push_back()
```

Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> numbers;

    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    return 0;
}
```

Now the vector contains:

```
10  20  30
```

How does it work?

```
numbers.push_back(10);
```

means:

Add 10 at the end of the vector.

Then:

```
numbers.push_back(20);
```

means:

Add 20 at the end.

So:

Initially:

```
[]
```

After `push_back(10)`:

```
[10]
```

After `push_back(20)`:

```
[10, 20]
```

After `push_back(30)`:

```
[10, 20, 30]
```

Display Vector Elements

We can access elements using indexes.

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> numbers;

    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    cout << numbers[0] << endl;
    cout << numbers[1] << endl;
    cout << numbers[2] << endl;

    return 0;
}
```

Output:

```
10
20
30
```

Remember:

Vector indexing starts from 0.

Value:	10	20	30
Index:	0	1	2

Using a Loop with Vector

Instead of writing:

```
cout << numbers[0];
cout << numbers[1];
cout << numbers[2];
```

we can use a loop.

```
for(int i = 0; i < numbers.size(); i++)
{
    cout << numbers[i] << endl;
}
```

Complete example:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> numbers;
```

```

    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    for(int i = 0; i < numbers.size(); i++)
    {
        cout << numbers[i] << endl;
    }

    return 0;
}

```

Output:

```

10
20
30

```

size() Function

`size()` tells us how many elements are currently present in the vector.

```

vector<int> numbers;

numbers.push_back(10);
numbers.push_back(20);
numbers.push_back(30);

cout << numbers.size();

```

Output:

```

3

```

If we add another element:

```

numbers.push_back(40);

```

Then:

```

numbers.size()

```

becomes:

```

4

```

So:

`size()` = current number of elements.

Removing the Last Element — `pop_back()`

To remove the last element:

```

pop_back()

```

Example:

```

vector<int> numbers = {10, 20, 30};

```

```
numbers.pop_back();
```

Before:

```
10 20 30
```

After:

```
10 20
```

Important:

`pop_back()` removes only the **last element**.

Accessing the First and Last Element

STL provides:

```
front()
```

for the first element.

```
back()
```

for the last element.

Example:

```
vector<int> numbers = {10, 20, 30, 40};  
  
cout << numbers.front() << endl;  
cout << numbers.back() << endl;
```

Output:

```
10  
40
```

Clearing a Vector

To remove all elements:

```
clear()
```

Example:

```
vector<int> numbers = {10, 20, 30};  
  
numbers.clear();  
  
cout << numbers.size();
```

Output:

```
0
```

The vector still exists, but it contains no elements.

Important Vector Functions

Function	Purpose
<code>push_back()</code>	Adds element at the end
<code>pop_back()</code>	Removes last element
<code>size()</code>	Returns number of elements
<code>front()</code>	Returns first element
<code>back()</code>	Returns last element
<code>clear()</code>	Removes all elements
<code>empty()</code>	Checks whether vector is empty
<code>at()</code>	Accesses an element with bounds checking

Example:

```
vector<int> numbers = {10, 20, 30};

cout << numbers.size();
cout << numbers.front();
cout << numbers.back();
```

empty()

`empty()` checks whether the vector contains any elements.

```
vector<int> numbers;

if(numbers.empty())
{
    cout << "Vector is empty";
}
```

Output:

Vector is empty

After:

```
numbers.push_back(10);
```

the vector is no longer empty.

Vector with Predefined Values

We don't always have to use `push_back()`.

We can directly initialize a vector:

```
vector<int> numbers = {10, 20, 30, 40, 50};
```

Example:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> marks = {80, 75, 90, 65, 88};
```

```

        for(int i = 0; i < marks.size(); i++)
        {
            cout << marks[i] << endl;
        }

        return 0;
    }

```

Range-Based For Loop

C++ also provides a simpler way to traverse a vector:

```

for(int x : numbers)
{
    cout << x << endl;
}

```

Example:

```

vector<int> numbers = {10, 20, 30, 40};

for(int x : numbers)
{
    cout << x << endl;
}

```

Output:

```

10
20
30
40

```

Here:

```
int x
```

temporarily receives each element.

Vector of Strings

Vector does not only store integers.

We can store strings:

```

vector<string> names;

names.push_back("Rahul");
names.push_back("Amit");
names.push_back("Priya");

```

Display:

```

for(string name : names)
{
    cout << name << endl;
}

```

Output:

Rahul
Amit
Priya

Vector of Objects

This is especially important for our **Student Management System**.

Suppose we have:

```
class Student
{
public:
    int rollNo;
    string name;
};
```

We can create:

```
vector<Student> students;
```

Now the vector can store multiple `Student` objects.

Example:

```
Student s1;
s1.rollNo = 101;
s1.name = "Rahul";

Student s2;
s2.rollNo = 102;
s2.name = "Amit";

students.push_back(s1);
students.push_back(s2);
```

Conceptually:

```
vector<Student>
```

Student 101 Rahul	Student 102 Amit
----------------------	---------------------

This is why STL becomes extremely useful in real projects.

What We Will Learn Next

After `vector`, we should **not jump directly to `map` or `set`**.

The next important topic is:

Iterators

We will understand:

```
begin()
end()
```

and:

```
vector<int>::iterator
```

Then we will connect iterators with STL algorithms:

```
sort()
find()
reverse()
```

After that:

```
vector
  ↓
Iterators
  ↓
Algorithms
  ↓
pair
  ↓
map
  ↓
set
  ↓
stack
  ↓
queue
  ↓
priority_queue
```

This sequence will make STL much easier to understand rather than memorizing functions.

Ok next notes

What is an Iterator?

An **iterator** is an object used to **access and move through the elements of an STL container**.

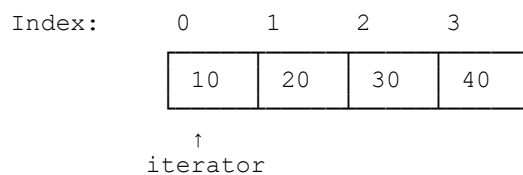
A simple way to understand it:

Iterator works somewhat like a pointer that moves through the elements of a container.

Suppose we have:

```
vector<int> numbers = {10, 20, 30, 40};
```

Conceptually:



The iterator can move from $10 \rightarrow 20 \rightarrow 30 \rightarrow 40$.

Why Do We Need Iterators?

You can access a vector using indexes:

```
numbers[0]
numbers[1]
numbers[2]
```

But STL has many containers that don't provide normal array-style indexing.

For example:

```
vector
list
set
map
```

Iterators provide a common way to traverse these containers.

So instead of depending on:

```
container[index]
```

we can use:

```
iterator
```

begin () and end ()

Two very important STL functions are:

```
begin()
end()
```

Suppose:

```
vector<int> numbers = {10, 20, 30, 40};
```

begin ()

```
numbers.begin()
```

points to the **first element**.

```
10  20  30  40
↑
begin()
```

end ()

```
numbers.end()
```

points to the position **just after the last element**.

```
10  20  30  40  ↑
                  end()
```

Important:

`end ()` does NOT point to 40. It points to the position after 40.

Creating an Iterator

Example:

```
vector<int>::iterator it;
```

Here:

```
vector<int>
  ↓
container contains integers

iterator
  ↓
type of iterator

it
  ↓
iterator variable
```

Complete example:

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> numbers = {10, 20, 30, 40};

    vector<int>::iterator it;

    return 0;
}
```

Assigning `begin()` to Iterator

We can write:

```
it = numbers.begin();
```

Now `it` points to the first element.

numbers

10	20	30	40
----	----	----	----

↑
`it`

Accessing the Value Using `*`

Just like a pointer, we can use `*` with an iterator.

```
cout << *it;
```

Output:

10

Why?

Because:

```
it
```

contains the position of the element.

And:

```
*it
```

means:

Give me the value at the position where `it` is pointing.

Moving the Iterator

We can move an iterator using:

```
it++;
```

Initially:

```
10  20  30  40
  ↑
  it
```

After:

```
it++;
```

it becomes:

```
10  20  30  40
      ↑
      it
```

Another:

```
it++;
```

Now:

```
10  20  30  40
          ↑
          it
```

Complete Iterator Example

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> numbers = {10, 20, 30, 40};

    vector<int>::iterator it;
```

```

    for(it = numbers.begin(); it != numbers.end(); it++)
    {
        cout << *it << endl;
    }

    return 0;
}

```

Output:

```

10
20
30
40

```

Let's understand this carefully.

```
it = numbers.begin();
```

Start from the first element.

```
it != numbers.end();
```

Continue until the iterator reaches the position after the last element.

```
it++
```

Move to the next element.

```
*it
```

Get the current element's value.

The Most Important Pattern in STL

You will see this pattern again and again:

```

for(auto it = container.begin(); it != container.end(); it++)
{
    cout << *it;
}

```

For example:

```

vector<int> numbers = {10, 20, 30, 40};

for(auto it = numbers.begin(); it != numbers.end(); it++)
{
    cout << *it << " ";
}

```

Output:

```
10 20 30 40
```

What is auto?

Instead of writing:

```
vector<int>::iterator it;
```

we can write:

```
auto it;
```

But `auto` needs an initial value so C++ can determine its type.

Correct:

```
auto it = numbers.begin();
```

C++ understands that `it` should be:

```
vector<int>::iterator
```

So this:

```
auto it = numbers.begin();
```

is equivalent to:

```
vector<int>::iterator it = numbers.begin();
```

Why is `auto` useful?

For complicated STL types, writing the complete type can become lengthy.

Therefore, modern C++ commonly uses:

```
auto
```

Iterator vs Index

Both can be used to traverse a vector.

Using index

```
for(int i = 0; i < numbers.size(); i++)  
{  
    cout << numbers[i] << endl;  
}
```

Using iterator

```
for(auto it = numbers.begin(); it != numbers.end(); it++)  
{  
    cout << *it << endl;  
}
```

Both can produce:

```
10  
20  
30  
40
```

But iterators become much more important when working with other STL containers such as `set` and `map`.

STL Algorithms

Now that we understand iterators, we can understand one of the most useful parts of STL:

Algorithms

Algorithms are ready-made functions that perform common operations on containers.

Examples:

```
sort()
find()
reverse()
count()
max_element()
min_element()
```

`sort()`

`sort()` is used to arrange elements in ascending or descending order.

First include:

```
#include <algorithm>
```

Example:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main()
{
    vector<int> numbers = {50, 20, 40, 10, 30};

    sort(numbers.begin(), numbers.end());

    for(int x : numbers)
    {
        cout << x << " ";
    }

    return 0;
}
```

Output:

```
10 20 30 40 50
```

Understand this:

```
sort(numbers.begin(), numbers.end());
```

means:

Sort the elements from the beginning of the vector to the end.

Sorting in Descending Order

We can use:

```
greater<int>()
```

Example:

```
sort(numbers.begin(), numbers.end(), greater<int>());
```

Complete:

```
vector<int> numbers = {50, 20, 40, 10, 30};  
sort(numbers.begin(), numbers.end(), greater<int>());
```

Result:

```
50 40 30 20 10
```

reverse()

`reverse()` reverses the order of elements.

```
vector<int> numbers = {10, 20, 30, 40};  
reverse(numbers.begin(), numbers.end());
```

Result:

```
40 30 20 10
```

find()

`find()` searches for an element.

Example:

```
vector<int> numbers = {10, 20, 30, 40};  
auto it = find(numbers.begin(), numbers.end(), 30);
```

We are asking:

Search for 30 from the beginning to the end.

If 30 exists, it points to 30.

We can check:

```
if(it != numbers.end())  
{  
    cout << "Element found";  
}  
else  
{
```

```
    cout << "Element not found";  
}
```

Complete example:

```
#include <iostream>  
#include <vector>  
#include <algorithm>  
using namespace std;  
  
int main()  
{  
    vector<int> numbers = {10, 20, 30, 40};  
  
    auto it = find(numbers.begin(), numbers.end(), 30);  
  
    if(it != numbers.end())  
    {  
        cout << "Element found";  
    }  
    else  
    {  
        cout << "Element not found";  
    }  
  
    return 0;  
}
```

Output:

Element found

Why Does `find()` Return an Iterator?

This is an important STL concept.

Suppose:

```
10  20  30  40  
      ↑  
     30
```

If `find()` finds 30, it can return its **position**.

Therefore:

```
auto it = find(...);
```

gives us an iterator.

Then:

```
*it
```

gives the actual value:

```
30
```

If the element isn't found:

```
it == numbers.end()
```

So this pattern is extremely important:

```
auto it = find(container.begin(), container.end(), value);

if(it != container.end())
{
    // found
}
else
{
    // not found
}
```

Important Algorithms

You should remember these initially:

Algorithm	Purpose
<code>sort()</code>	Sort elements
<code>find()</code>	Search for an element
<code>reverse()</code>	Reverse elements
<code>count()</code>	Count occurrences
<code>max_element()</code>	Find maximum
<code>min_element()</code>	Find minimum

Example:

```
vector<int> numbers = {10, 20, 30, 20, 40};
```

Count 20:

```
int result = count(numbers.begin(), numbers.end(), 20);
```

Result:

2

Find maximum:

```
auto it = max_element(numbers.begin(), numbers.end());
```

Then:

```
cout << *it;
```

Output:

40

Very Important STL Pattern

You should now start recognizing this:

```
container.begin()
```

means:

Start position

and:

```
container.end()
```

means:

Position just after the last element

Therefore many STL algorithms look like:

```
algorithm(container.begin(), container.end());
```

For example:

```
sort(numbers.begin(), numbers.end());  
reverse(numbers.begin(), numbers.end());  
find(numbers.begin(), numbers.end(), 30);  
count(numbers.begin(), numbers.end(), 20);
```

This is one of the **most important patterns in STL**.

Practice Questions

After teaching this topic, give the student these exercises.

Exercise 1

Create a vector:

```
10 50 20 40 30
```

Sort it in ascending order.

Exercise 2

Sort the same vector in descending order.

Exercise 3

Search for 40 using `find()`.

Exercise 4

Reverse the vector using `reverse()`.

Exercise 5

Create:

```
10 20 10 30 10 40
```

Count how many times 10 occurs.

Exercise 6

Find the maximum and minimum element.

Exercise 7

Create a vector of student marks:

85 72 91 68 77

Sort the marks and display them.